

BARD

BEHAVIOURALLY ADAPTIVE REACTIVE DIALOGUE

ERIMENTHA — AN LLM-DRIVEN RPG DEMO · A LOCAL LANGUAGE MODEL REWRITES NPC DIALOGUE EVERY IN-GAME NIGHT BASED ON WHAT THE PLAYER DID THAT DAY

Godot 4.6

GDScript

Python 3 Pipeline

Ollama · Gemma 4 E4B

Local GPU Inference

Academic Capstone 2026

THE LLM PIPELINE

Every night a **Python subprocess** reads the day's game state and prompts a locally-running LLM to write fresh dialogue for each named NPC — no cloud, no API keys, no internet required.

```
Player sleeps at Inn
→ end_day() writes game_state.json
→ Godot spawns end_of_day.py subprocess
→ Pipeline reads kills, bounties, flags, NPC memory
→ Ollama (Gemma 4 E4B) generates dialogue JSON
→ Validates schema → LLM repair if malformed → fallback
→ Writes dialogue/{npc_id}_day{N}.json
→ LLM extracts one NPC recollection fact
→ Writes pipeline_ready.flag
→ Godot detects flag → reloads dialogue → go to town
```

Godot polls for sentinel flag files every 3 seconds — no blocking threads. A progress bar animates as each NPC is processed. The pipeline writes `pipeline_ready.flag`, `pipeline_failed.flag`, or `pipeline_crashed.flag` depending on outcome.

NPC MEMORY SYSTEM

NPCs don't just react to today — they *remember*. After generating each NPC's dialogue, the pipeline makes a second LLM call to extract one subjective recollection and appends it to `game_state.json`. Future prompts inject these facts, building narrative persistence across days.

"Aerin returned today with slime cores — seven of them. She mentioned the pack was larger than expected."
— Example recollection stored in `npc_facts` after Day 3

Context injected into every NPC prompt: kill reports for all four monster families · active and completed bounty status · `met_*` and `first_bounty_*` interaction flags · all stored recollection facts · circulating rumors from the Chronicle · town lore + NPC personality variant

CHRONICLE PIPELINE

Triggered with **Ctrl+R** in town — does not advance the day. Reads the full `monsters_killed_history` and generates a narrative chronicle of the week's events, then derives **rumors** from the player's notable deeds. Rumors are injected into all future end-of-day prompts, giving NPCs indirect awareness of events they didn't witness directly.

Rumor attribution 75% named · 25% anonymous
Rumor pool cap 10 entries (oldest pruned)
Output chronicles/week_N.json · rumors.json

BOUNTY SYSTEM

Contracts are scoped to specific monster types and zones — giving the LLM rich, precise data to reference each night when generating NPC dialogue.

```
bounty_pool.json — 36 static contracts
→ Daily refresh: min(day, 3) contracts shown
→ Player accepts → tracked in active_bounties
→ Zone kills auto-increment the counter
→ Turn in at Guild Commander → earn Scripts
→ end_day() writes completed data → LLM reads it
```

Small 10 Scripts Zone A Orcs + Plants
Medium 25 Scripts Zone B Vampires
Large 50 Scripts Zone C Slimes

TECHNOLOGY STACK

Game Engine Godot 4.6 · GDScript
LLM Runtime Ollama — local GPU inference
LLM Model Gemma 4 E4B
Pipeline Python 3 (subprocess IPC)
Interchange JSON + sentinel flag files
UI Architecture 100% GDScript · signal-driven
Resolution 1920×1080 · Forward+ renderer

DIALOGUE ARCHITECTURE

LLM-generated content is surgically **merged** with hardcoded role menus at runtime — preserving reliability while the LLM controls all narrative content.

Hardcoded: Sleep · Buy Axe · Upgrade Weapons · Bounty Board · Turn In · Goodbye
LLM-generated: greeting node + all branching conversation trees

Validation enforces: `greeting` + `farewell` required, no broken `next` references, no dead-end cycles. A failed validation triggers an **LLM repair call** before falling back to the previous day's file, then to static fallbacks in `pipeline/fallbacks/`.

World generation world_gen.py — run once
NPC variants per role 3 personality variants (A/B/C)
Named NPCs Innkeeper · Blacksmith · Guild Commander